

BÁO CÁO THỰC HÀNH ASP.NET CORE MVC

Lab 03: Layout, Partial View, Tag Helpers & Model Binding

Họ và tên: Lê Kim Hùng

MSSV: 22110068

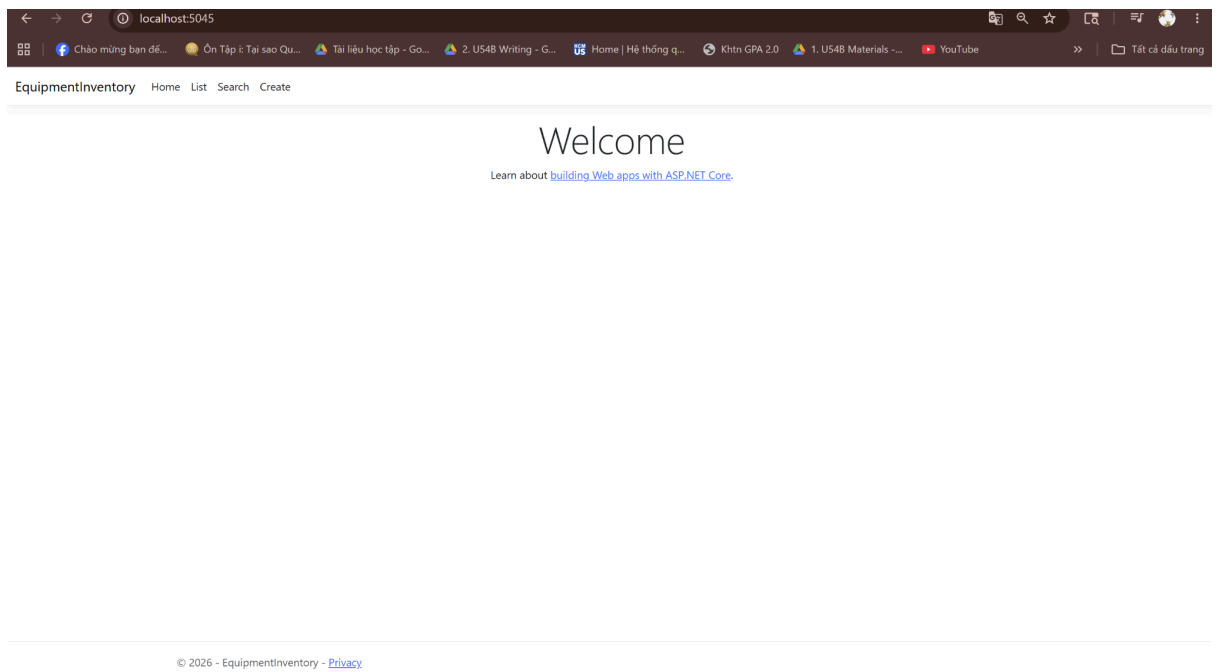
Giảng viên hướng dẫn: Ts. Trần Anh Tuấn

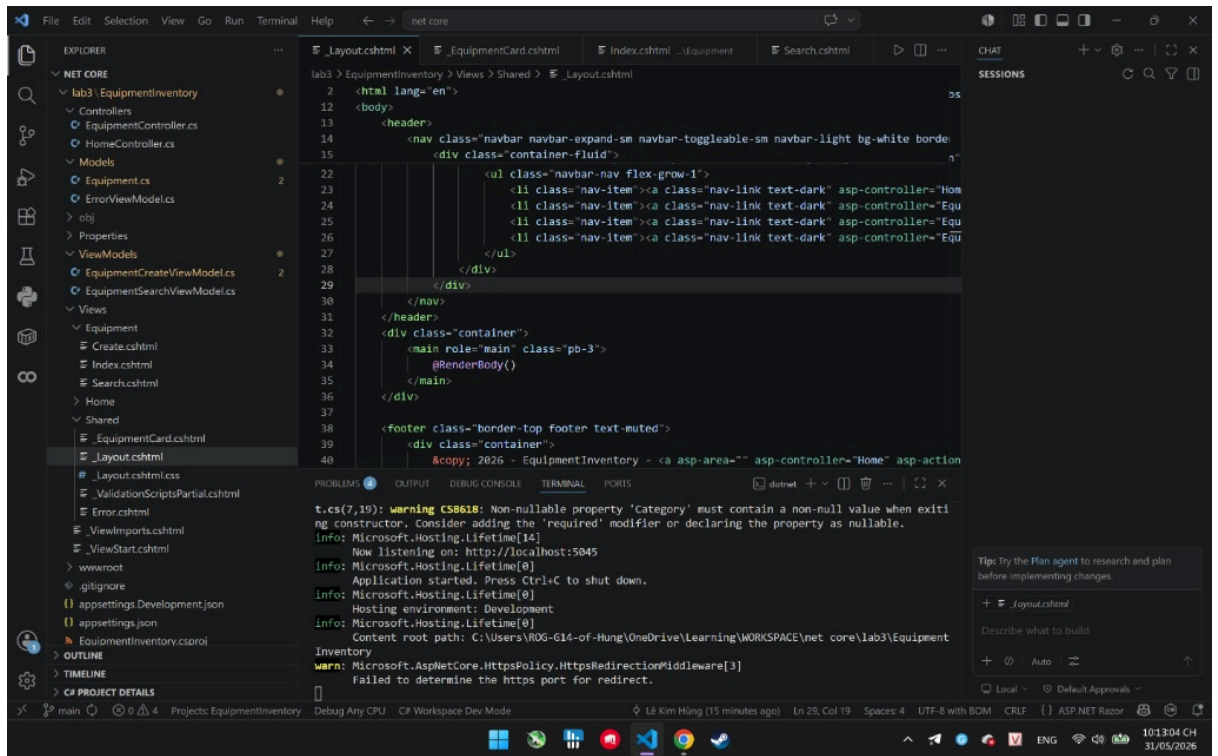
CÂU 1: TRIỂN KHAI ỨNG DỤNG MINI EQUIPMENT INVENTORY MVC

Ứng dụng quản lý thiết bị được nâng cấp cấu trúc chuẩn MVC, áp dụng các kỹ thuật tái sử dụng giao diện và xử lý form an toàn. Dưới đây là diễn giải các thành phần đã thực hiện:

1. Layout dùng chung và Menu điều hướng

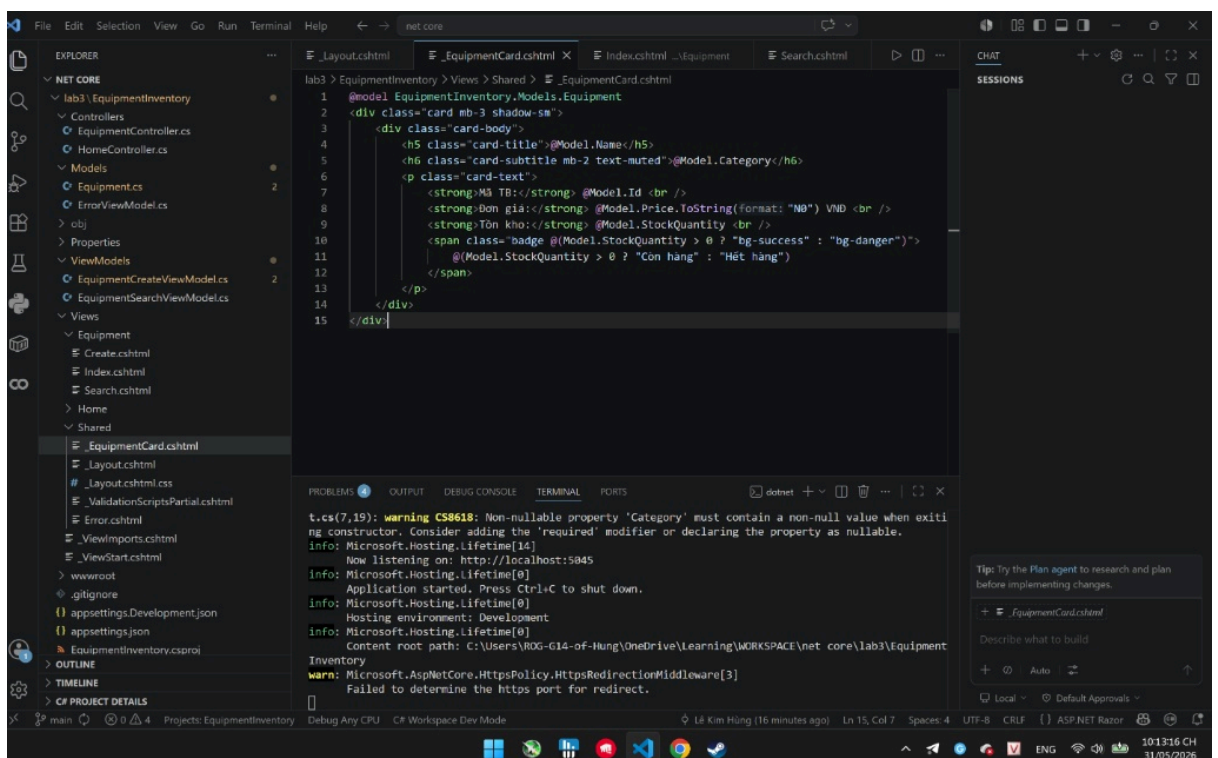
Hệ thống sử dụng một file `_Layout.cshtml` duy nhất làm bộ khung. Menu điều hướng được xây dựng sử dụng Tag Helpers (`asp-controller`, `asp-action`) bao gồm các mục: Home, List, Stats, Search, Create.





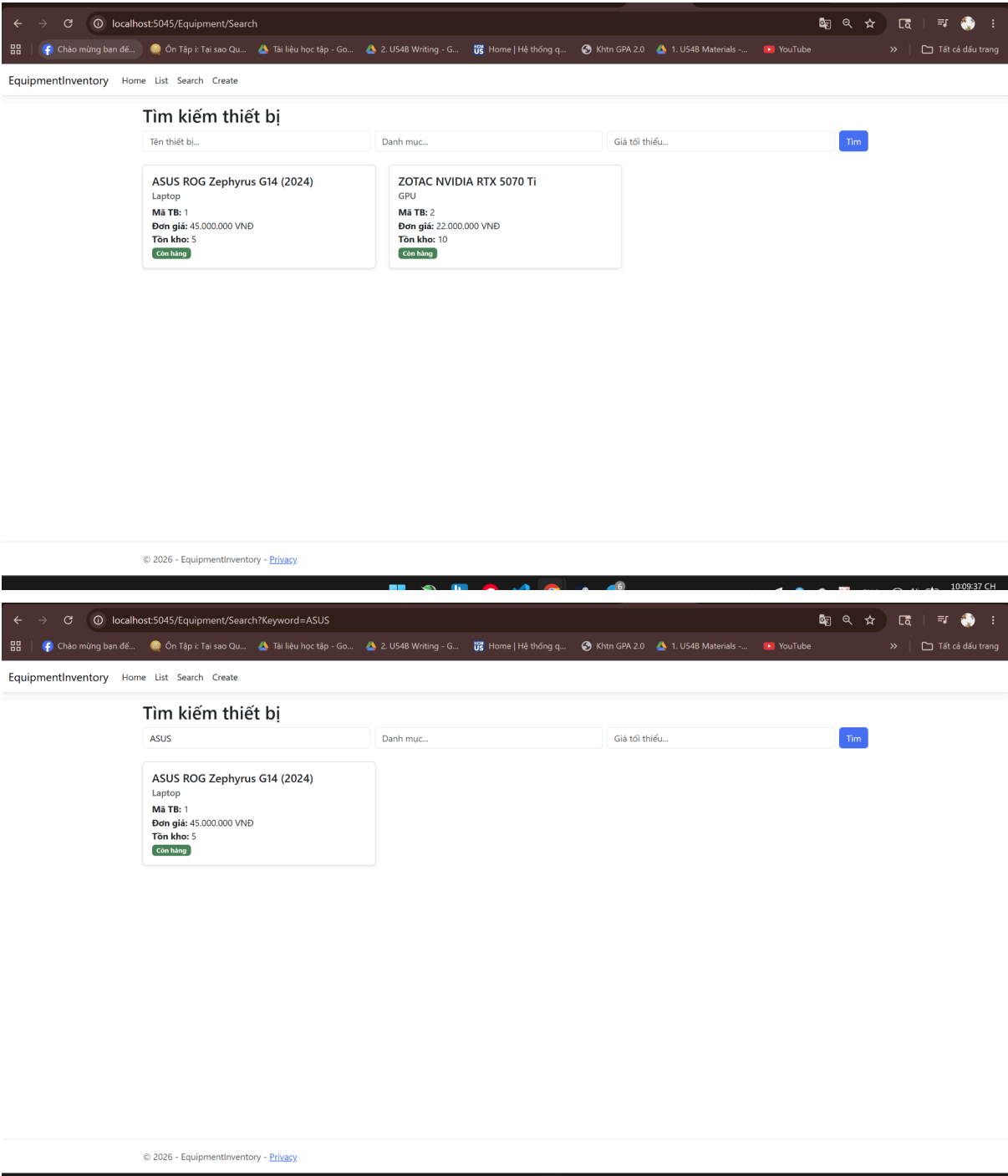
2. Partial View tái sử dụng giao diện

Phần hiển thị thông tin chi tiết của một thiết bị (Tên, Danh mục, Đơn giá, Số lượng tồn, Trạng thái) được tách riêng vào file `Views/Shared/_EquipmentCard.cshtml`. Partial View này được gọi bằng thẻ `<partial name="_EquipmentCard" model="item" />` để dùng chung cho cả trang Danh sách và trang Tìm kiếm.



3. Trang Tìm kiếm (GET Form & Query String)

Xây dựng ViewModel `EquipmentSearchViewModel` và View Search chứa form tìm kiếm sử dụng method GET. Người dùng có thể tìm theo tên (Keyword), danh mục (Category) và mức giá tối thiểu (MinPrice). Kết quả được hiển thị ngay bên dưới form thông qua việc tái sử dụng thẻ Partial View `_EquipmentCard`.



4. Trang Thêm mới Thiết bị (POST Form & Model Binding)

Trang Create sử dụng một ViewModel riêng là **EquipmentCreateViewModel**. Form POST được thiết kế với Tag Helpers (**asp-for**, **asp-validation-for**) ánh xạ trực tiếp đến các property: Tên thiết bị, Danh mục, Đơn giá, Số lượng tồn.

Thêm Thiết Bị Mới

Tên thiết bị
Lenovo Legion 7i

Danh mục
Laptop

Đơn giá (VND)
50000000

Số lượng tồn
3

Thêm mới

© 2026 - EquipmentInventory - Privacy

```
1 using System.ComponentModel.DataAnnotations;
2
3 namespace EquipmentInventory.ViewModels;
4
5 public class EquipmentCreateViewModel
6 {
7     [Required(ErrorMessage = "Tên thiết bị không được để trống")]
8     public string Name { get; set; }
9
10    [Required(ErrorMessage = "Danh mục không được để trống")]
11    public string Category { get; set; }
12
13    [Range(1, double.MaxValue, ErrorMessage = "Giá phải lớn hơn 0")]
14    public decimal Price { get; set; }
15
16    [Range(0, int.MaxValue, ErrorMessage = "Số lượng tồn không được âm")]
17    public int StockQuantity { get; set; }
18 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

t.cs(7,10): warning CS8618: Non-nullable property 'Category' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the property as nullable.

info: Microsoft.Hosting.Lifetime[14] Now listening on: http://localhost:5045

info: Microsoft.Hosting.Lifetime[0] Application started. Press Ctrl+C to shut down.

info: Microsoft.Hosting.Lifetime[0] Hosting environment: Development

info: Microsoft.Hosting.Lifetime[0] Content root path: C:\Users\ROG-G14-of-Hung\OneDrive\Learning\WORKSPACE\net core\lab3\EquipmentInventory

warn: Microsoft.AspNetCore.HttpsPolicy.HttpsRedirectionMiddleware[3] Failed to determine the https port for redirect.

Tip: Try the Plan agent to research and plan before implementing changes.

+ EquipmentCreateViewModel.cs

Describe what to build

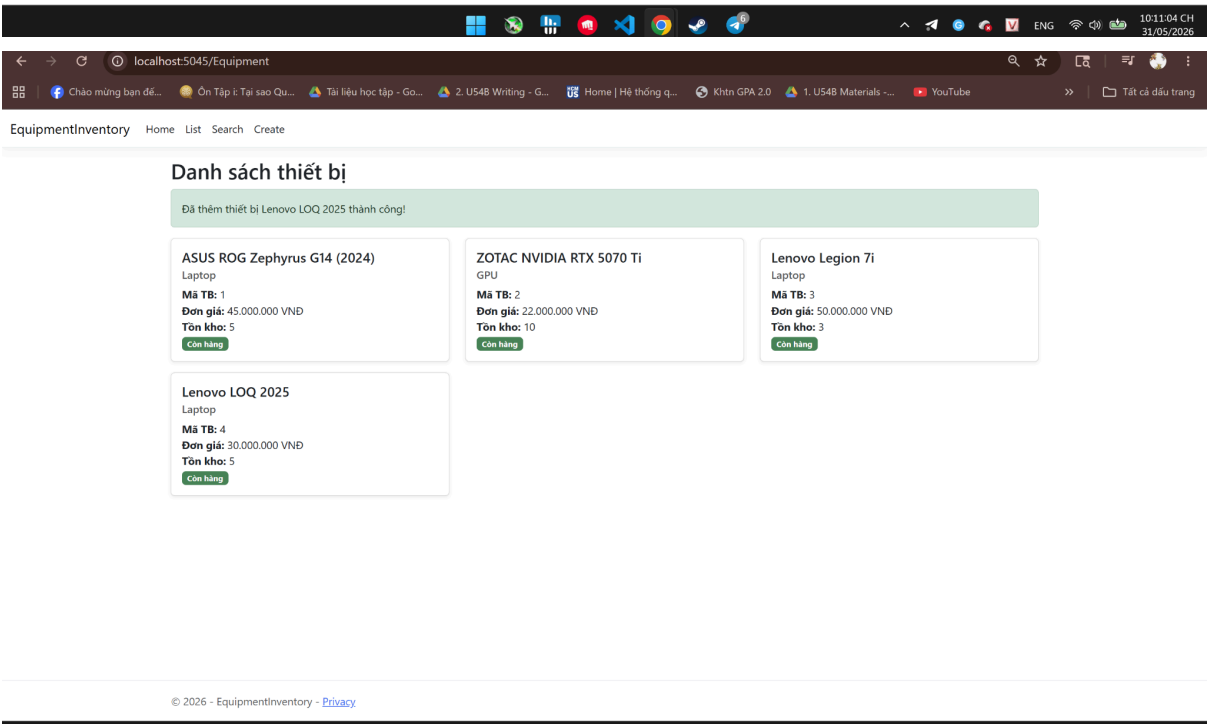
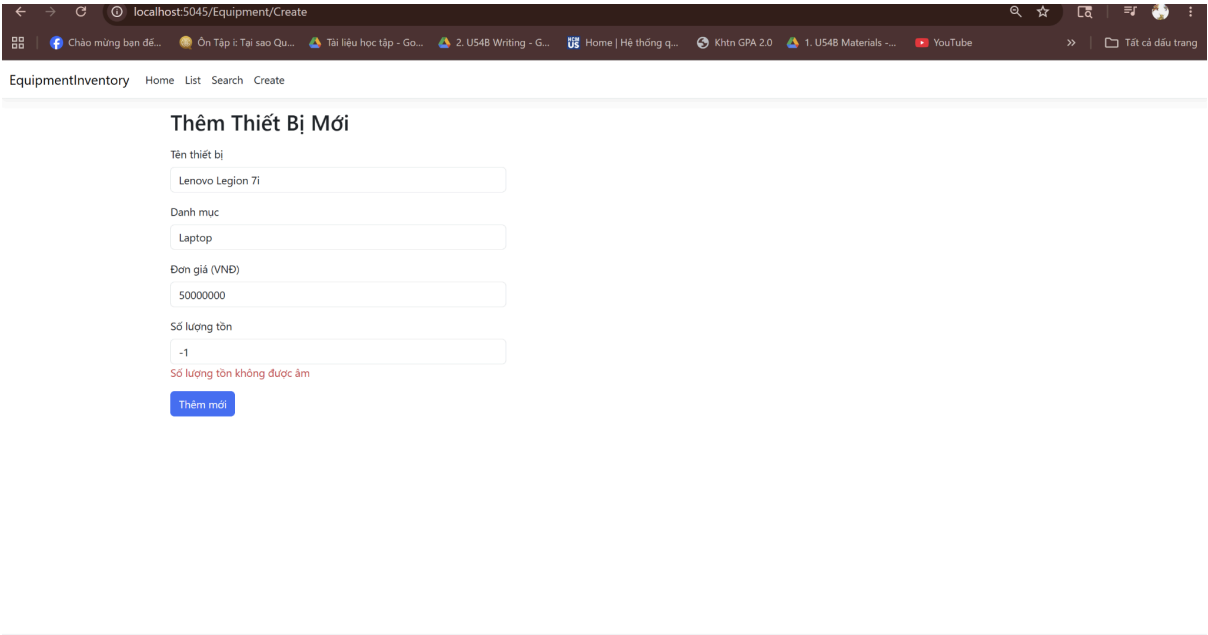
+ Auto

Local Default Approvals

5. Validation và Thông báo thành công (TempData)

Controller kiểm tra tính hợp lệ của dữ liệu thông qua **ModelState.IsValid**. Nếu nhập sai (ví dụ: để trống tên, giá trị âm), form sẽ báo lỗi chữ đỏ. Nếu hợp lệ, hệ thống dùng

`RedirectToAction("Index")` chuyển hướng về trang danh sách và hiển thị thông báo "Thêm thiết bị thành công" màu xanh bằng `TempData`.

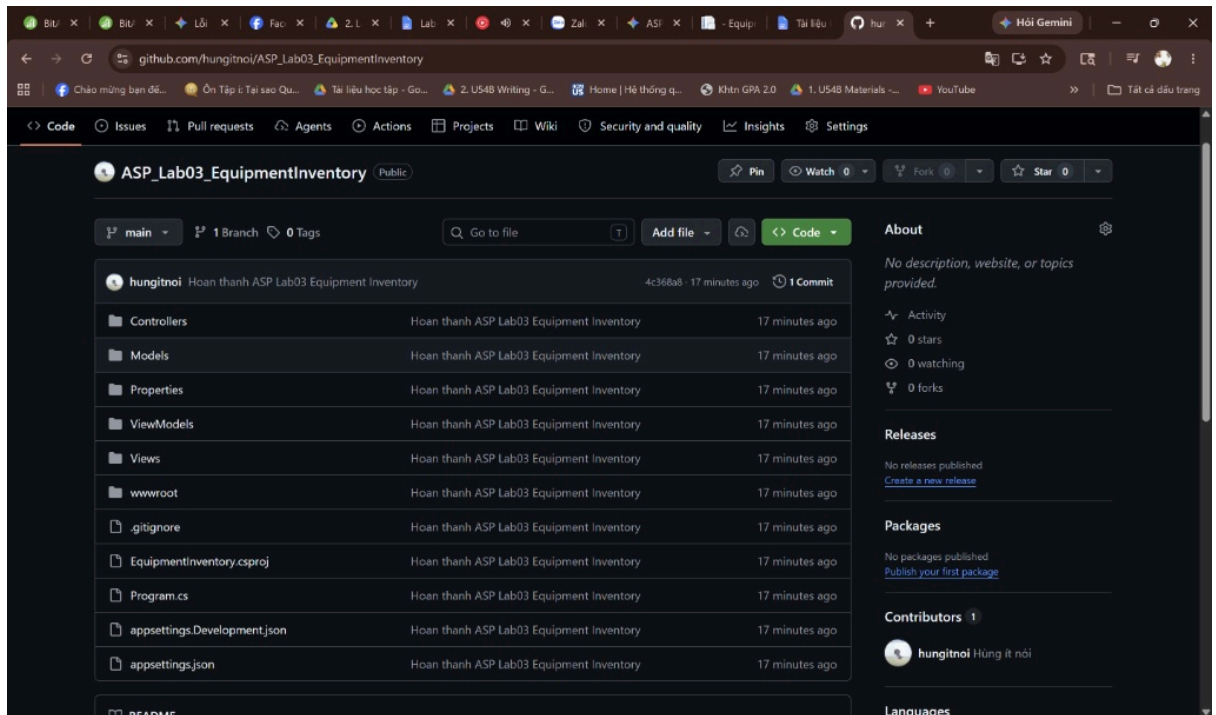


6. Quản lý mã nguồn (GitHub)

Toàn bộ source code của ứng dụng đã được commit và đẩy lên GitHub.

- **Link GitHub Repository:**

https://github.com/hungitnoi/ASP_Lab03_EquipmentInventory



CÂU 2: TRẢ LỜI CÂU HỎI PROBLEM SOLVING

1. Layout giúp giải quyết vấn đề gì khi website có nhiều trang? Nếu không dùng Layout, việc bảo trì giao diện sẽ gặp khó khăn gì? Layout cung cấp một bộ khung giao diện chung (header, menu, footer, link CSS/JS) cho toàn bộ ứng dụng. Nếu không dùng Layout, trong dự án Equipment Inventory, khi muốn thêm mục "Stats" vào thanh menu, em sẽ phải mở từng file `Index.cshtml`, `Search.cshtml`, `Create.cshtml` để sửa tay. Việc này gây lặp code, tốn thời gian và cực kỳ dễ xảy ra sai sót không đồng bộ.

2. Partial View trong bài của em dùng để tái sử dụng phần giao diện nào? Vì sao phần đó nên được tách ra Partial View? Em dùng Partial View (`_EquipmentCard.cshtml`) để đóng gói khối giao diện hiển thị 1 thẻ thiết bị (gồm Tên, Mã, Giá, Tồn kho). Nên tách ra vì khối giao diện này được sử dụng lặp lại rất nhiều lần: xuất hiện trong vòng lặp `foreach` của trang danh sách thiết bị (List) và cũng được dùng lại y hệt trong trang hiển thị kết quả tìm kiếm (Search).

3. Vì sao nên dùng Tag Helpers như `asp-controller`, `asp-action`, `asp-route-id` thay vì viết URL cứng bằng `href="/..."`? Tag Helpers sẽ tự động sinh URL dựa trên bảng định tuyến (Routing Table). Ví dụ: `asp-controller="Equipment" asp-action="Create"`. Nếu sau này hệ thống đổi route từ `/Equipment/Create` thành `/Inventory/Add`, thẻ Tag Helper sẽ tự cập nhật URL đúng trên toàn bộ view. Nếu viết cứng bằng

`href="/Equipment/Create"`, đường link sẽ bị gãy (404) và em phải đi tìm sửa thủ công khắp nơi.

4. Form tìm kiếm nên dùng GET hay POST? Vì sao? Nên dùng GET. Vì thao tác tìm kiếm thiết bị là *Idempotent* (chỉ lấy dữ liệu ra xem, không làm thay đổi hay thêm bớt dữ liệu trên server). Dùng GET sẽ đưa tham số lên URL (ví dụ: `?Keyword=RTX`), giúp người dùng dễ dàng lưu bookmark hoặc gửi link kết quả tìm kiếm cho người khác.

5. Form thêm dữ liệu nên dùng GET hay POST? Vì sao? Bắt buộc dùng POST. Vì thao tác Create làm thay đổi trạng thái hệ thống (tạo ra một thiết bị mới). Dùng POST giúp ẩn dữ liệu khỏi URL, cho phép gửi dữ liệu dung lượng lớn, và quan trọng nhất là trình duyệt sẽ cảnh báo nếu người dùng lỡ nhấn F5 (refresh), giúp tránh việc insert trùng lặp một thiết bị nhiều lần.

6. Model Binding hoạt động như thế nào trong form Create của em? Hãy giải thích mối liên hệ giữa `asp-for`, tên property trong ViewModel và dữ liệu action nhận được. Model Binding tự động trích xuất dữ liệu từ HTTP Request gửi lên và ánh xạ vào các property của `EquipmentCreateViewModel`. Khi dùng `asp-for="Price"`, Tag Helper tự render HTML `<input name="Price">`. Khi submit form, MVC sẽ dò tìm key "Price" trong dữ liệu POST và tự động chuyển đổi kiểu dữ liệu, gán giá trị đó vào property `Price` của object `model` trong tham số của Action `Create`.

7. Nếu submit form nhưng action nhận giá trị null hoặc 0, em sẽ kiểm tra những điểm nào? Em sẽ kiểm tra:

- Thuộc tính `name` của thẻ `<input>` trong HTML có khớp chính xác với tên property trong ViewModel không (ví dụ gõ nhầm thành `name="Prcie"`).
- Thẻ `<form>` đã có thuộc tính `method="post"` chưa.
- Action POST trong Controller đã định nghĩa đúng tham số nhận là `EquipmentCreateViewModel` hay chưa.

8. Vì sao cần `ModelState.IsValid` trước khi thêm dữ liệu mới? Để bảo mật và đảm bảo tính toàn vẹn dữ liệu ở phía Server. Dù giao diện HTML có chặn người dùng nhập sai, kẻ gian vẫn có thể dùng các công cụ như Postman để gửi thẳng request chứa dữ liệu rác lên. `ModelState.IsValid` dựa vào các DataAnnotations (như `[Required]`, `[Range]`) để chặn lại những thiết bị không có tên hoặc có giá âm trước khi ghi vào hệ thống.

9. Vì sao sau khi thêm dữ liệu thành công nên dùng `RedirectToAction()` thay vì `return View()` trực tiếp? Để giải quyết vấn đề PRG (Post-Redirect-Get). Nếu `return View()` trực tiếp sau khi thêm thiết bị, URL ở trình duyệt vẫn lưu trạng thái của request POST. Nếu người dùng vô tình nhấn F5, trình duyệt sẽ gửi lại form đó, tạo ra thiết bị trùng lặp. `RedirectToAction("Index")` ép trình duyệt ngắt luồng POST và thực hiện một request GET mới hoàn toàn sang trang danh sách.

10. Vì sao dữ liệu mới thêm vào bị mất khi tắt app và chạy lại? Nếu muốn dữ liệu không mất, các tuần sau cần bổ sung thành phần nào? Dữ liệu đang được lưu tạm thời trên RAM thông qua biến `static List<Equipment>`. Khi tiến trình .NET dừng, RAM bị

giải phóng nên dữ liệu biến mất. Để dữ liệu tồn tại vĩnh viễn, các tuần sau cần bổ sung một Hệ quản trị cơ sở dữ liệu (như SQL Server/PostgreSQL) và dùng ORM như Entity Framework Core để ghi dữ liệu xuống ổ cứng.